

Automated Reformulation of Specifications by Safe Delay of Constraints

Marco Cadoli and Toni Mancini

Dipartimento di Informatica e Sistemistica
Università di Roma "La Sapienza"
Via Salaria 113, I-00198 Roma, Italy
cadoli | tmancini @di s. uni roma1. it

Abstract. In this paper we propose a form of reasoning on problem specifications, with the goal of reformulating them so that they are more efficiently solvable. The reformulation technique highlights constraints that can be safely "delayed", and solved afterwards. Our main contribution is the characterization (with soundness proof) of safe-delay constraints with respect to a criterion on the specification, thus obtaining a mechanism for the automated reformulation of specifications applicable to a great variety of problems, e.g., graph coloring and job shop scheduling. This is an advancement with respect to the forms of reasoning done by state-of-the-art-systems, which typically just detect linearity of specifications. Another contribution is a preliminary experimentation on the effectiveness of the proposed technique, which reveals promising time savings.

1 Introduction

Several systems and languages for the solution of constraint problems adopt a clear separation between the specification of a problem, e.g., graph three-coloring, and its instance, e.g., a graph. On top of that, they use a two-level architecture for finding solutions: the specification is instantiated against the instance, and then an appropriate solver is invoked. Examples of systems of such kind are *ampl* [10], *opl* [17], *dlv* [8], *smodels* [16], and *np-spec* [4].

The major benefit of this separation is the decoupling of the solver from the specification. Ideally, the programmer can focus only on the specification of the problem, without committing a priori to a specific solver. In fact, some systems, e.g., *ampl*, are able to translate –at the request of the user– a specification in various formats, suitable for different solvers.

Some systems go one step further, and offer a limited form of reasoning on the specification, with the goal of choosing the most appropriate solver for a problem. As an example, the *opl* system checks whether a specification contains only linear constraints and objective function, and in this case invokes an integer linear programming solver (typically very efficient); otherwise, it uses a constraint programming solver.

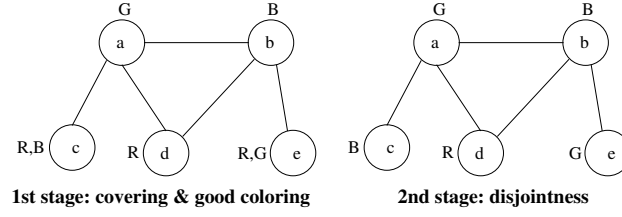


Fig. 1. Delaying the disjointness constraint in 3-coloring

Our long-term goal is to propose a more sophisticated architecture, in which reasoning on the specification is more complex. In particular, we propose to reformulate the specification so that the later stages of computation, i.e., instantiation and solution, are more efficient. We get inspiration from the relational database technology, since it is well-known that reformulating queries –independently on the database– may result in greater efficiency. As an example, making selections as soon as possible is a simple heuristic that typically allows to decrease the number of accesses to disk (cf., e.g., [1]).

Reformulation is quite difficult in general: a specification is essentially a formula in second-order logic, and it is well-known that the equivalence problem is undecidable already in the first-order case [3]. For this reason we limit our attention on restricted forms of reformulation, and, more specifically, in this paper we focus on a reformulation that selects constraints that can be safely “delayed”, and solved afterwards.

The NP-complete graph k -coloring problem offers a simple example of a constraint of this kind. The problem amounts to find an assignment of nodes to k colors such that:

- Each node has at least one color (covering);
- Each node has at most one color (disjointness);
- Adjacent nodes have different colors (good coloring).

For each instance of the problem, if we obtain a solution neglecting the disjointness constraint, we can always choose for each node one of its colors in an arbitrary way in a later stage (cf. Fig. 1). We call a constraint with this property a safe-delay constraint. It is interesting to note that the standard DIMACS formulation in SAT of k -coloring omits the disjointness constraint.

Of course not all constraints are safe-delay: as an example, both the covering and the good coloring constraints are not. Intuitively, identifying the set of constraints of a specification which are safe-delay may lead to several advantages:

- The instantiation phase will typically be faster, since safe-delay constraints are not taken into account. As an example, let’s assume we want to use a SAT solver (after instantiation) for the solution of k -coloring on a graph with n nodes and e edges. The SAT instance encoding the k -coloring instance –in the obvious way, cf., e.g., [12]– has $n \cdot k$ propositional variables, and a number of clauses which is $n, n \cdot k \cdot (k - 1) / 2$, and $e \cdot k$ for covering, disjointness, and

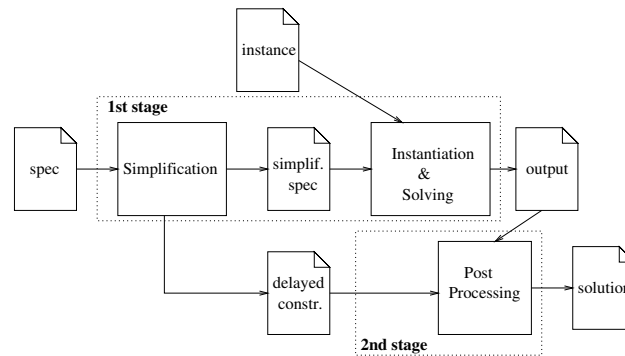


Fig. 2. Reformulation architecture

good coloring, respectively. If we delay disjointness, $n \cdot k \cdot (k - 1) / 2$ clauses must not be generated.

- Solving the simplified problem, i.e., the one without disjointness, might be easier than the original formulation for some classes of solvers. In our (even if preliminary) experiments, using a SAT solver, we obtained a fairly consistent (in some cases, more than one order of magnitude) speed-up for hard instances of various problems, e.g., graph coloring and job shop scheduling. On top of that, we implicitly obtain several good solutions. The approach seems promising for some classes of instances even when state-of-the-art solvers for integer linear programming like `cplex` are used (cf. Sec. 5).
- Ad hoc efficient methods for solving delayed constraints may exist. In fact, the problem of choosing only one color for the nodes with more than one color is $O(n)$.

The architecture we propose is illustrated in Fig. 2 and can be applied to any system which separates the instance from the specification. It is in some sense similar to the well-known divide and conquer technique, but rather than dividing the instance, we divide the constraints. In general, the first stage will be more computationally expensive than the second one, which, in our proposal, will always be doable in polynomial time.

The goal of this paper is to understand in which cases a constraint is safe-delay. Our main contribution is the characterization of safe-delay constraints with respect to a criterion on the specification. This allows us to obtain a mechanism for the automated reformulation of a specification (shown in Sec. 3) that can be applied to a great variety of problems, including the so-called functional ones. Another contribution is an experimentation on the effectiveness of the proposed reformulation. A preliminary discussion on the adopted experimental methodology is given in Sec. 4. The experiments are shown in Sec. 5, and have been done on both benchmark and randomly generated instances, using both a SAT solver and state-of-the-art linear and constraint programming solvers. Finally, conclusions and related work are described in Sec. 6.

2 Preliminaries

In this paper, we use existential second-order logic (ESO) for the specification of problems, which allows to represent all search problems in the complexity class NP [9]. The use of ESO as a modelling language for problem specifications is common in the database literature, but unusual in constraint programming, therefore few comments are in order. Constraint modelling systems like those mentioned in Sec. 1 have a richer syntax and more complex constructs, and we plan to eventually move from ESO to such languages. For the moment, we claim that studying the simplified scenario is a mandatory starting point for more complex investigations, and that our results can serve as a basis for reformulating specifications written in higher-level languages. In Sec. 4 we discuss further our choice. Coherently with all state-of-the-art systems, we represent an instance of a problem by means of a relational database. All constants appearing in a database are uninterpreted, i.e., they don't have a specific meaning.

In the following, σ denotes a fixed set of relational symbols not including equality "=", and $S_1, \dots, S_h$ denote variables ranging over relational symbols distinct from those in $\sigma \cup \{=\}$. By Fagin's theorem [9], any collection D of finite databases over σ is recognizable in NP time if and only if it is defined by an ESO formula of the kind:

$$\exists S_1, \dots, S_h \varphi, \quad (1)$$

where $S_1, \dots, S_h$ are relational variables of various arities and φ is a function-free first-order formula containing occurrences of relational symbols from $\sigma \cup \{S_1, \dots, S_h\} \cup \{=\}$. The symbol "=" is always interpreted in the obvious way, i.e., as "identity". A database D is in D if and only if there is a list of relations $\Sigma_1, \dots, \Sigma_h$ (matching the list of relational variables $S_1, \dots, S_h$) which, along with D , satisfies formula (1), i.e., such that $(D, \Sigma_1, \dots, \Sigma_h) \models \varphi$. The tuples of $\Sigma_1, \dots, \Sigma_h$ must take elements from the Herbrand universe of D , i.e., the set of constant symbols occurring in it.

Example 1. In the "three-coloring" NP-complete decision problem (cf. [13, Prob. GT4, p. 191]) the input is a graph, and the question is whether it is possible to give each of its nodes one out of three colors (red, green, and blue), in such a way that adjacent nodes (not including self-loops) are never colored the same way. The question can be easily specified as an ESO formula ψ over a binary relation edge :

$$\exists \text{RGB} \quad \exists X \quad R(X) \wedge G(X) \wedge B(X) \wedge \quad (2)$$

$$\exists X \quad R(X) \rightarrow \neg G(X) \wedge \quad (3)$$

$$\exists X \quad R(X) \rightarrow \neg B(X) \wedge \quad (4)$$

$$\exists X \quad B(X) \rightarrow \neg G(X) \wedge \quad (5)$$

$$\exists X \exists Y \quad X \neq Y \wedge R(X) \wedge R(Y) \rightarrow \neg \text{edge}(X, Y) \wedge \quad (6)$$

$$\exists X \exists Y \quad X \neq Y \wedge G(X) \wedge G(Y) \rightarrow \neg \text{edge}(X, Y) \wedge \quad (7)$$

$$\exists X \exists Y \quad X \neq Y \wedge B(X) \wedge B(Y) \rightarrow \neg \text{edge}(X, Y), \quad (8)$$

where clauses (2), (3-5), and (6-8) represent the covering, disjointness, and good coloring constraints, respectively. Referring to the graph in Fig. 1, the Herbrand universe is the set $\{a, b, c, d, e\}$, the input database has only one relation, i.e., edge, which has five tuples (one for each edge). Formula ψ is satisfied if R , G , and B are assigned to, e.g., the following relations (cf. Fig. 1, right):

R	G	B
$\begin{bmatrix} d \\ \end{bmatrix}$	$\begin{bmatrix} a \\ e \end{bmatrix}$	$\begin{bmatrix} b \\ c \end{bmatrix}$

In what follows, existentially quantified predicates (like R , G , and B) will be called guessed, and the set of tuples from the Herbrand universe they take will be called their extension and denoted with $\text{ext}()$. As an example, $\text{ext}(G) = \{a, e\}$ in the previous example. The symbol $\text{ext}()$ will be used also for any first-order formula with one free variable. An interpretation will be sometimes denoted as the aggregate of several extensions.

3 Reformulation

In this section we show sufficient conditions for constraints of a specification to be safe-delay. We refer to the architecture of Fig. 2, with some general assumptions:

1. As shown in Fig. 1, the output of the first stage of computation may – implicitly – contain several solutions. In the second stage we do not want to compute all of them, but just to arbitrarily select one.
2. The second stage of computation can only shrink the extension of a guessed predicate. Fig. 3 represents the extensions of the red predicate in the first ($R^\square$) and second (R) stages of Fig. 1 ($\text{ext}(B)$ and $\text{ext}(G)$ are unchanged). This assumption is coherent with the way most algorithms for constraint satisfaction operate: each variable has an associated finite domain, from which values are progressively eliminated, until a satisfying assignment is found.

Identification of safe-delay constraints requires reasoning on the whole specification, taking into account relations between guessed and database predicates. For the sake of simplicity, we will initially focus our attention on a single monadic guessed predicate, trying to figure out which constraints concerning it can be delayed. Afterwards, we extend our results to sets of monadic guessed predicates, then to binary predicates.

3.1 Single monadic predicate

We refer to the 3-coloring specification of Example 1, focusing on the guessed predicate R , and trying to find an intuitive explanation for the fact that clauses (3-4) can be delayed. We immediately note that clauses in the specification can be partitioned into three subsets:

NO: R does not occur in them, i.e., (5,7,8);

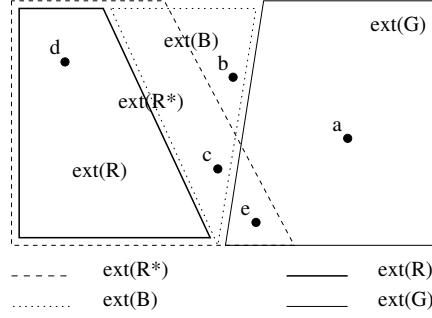


Fig. 3. Extensions for the 3-coloring spec

NEG: R occurs negatively in them, i.e., (3,4,6);

POS: R occurs positively in them, i.e., (2).

Neither **NO** nor **NEG** clauses can be violated by shrinking the extension of R . Such constraints will be called *safe-forget* for R , because if we decide to process (and satisfy) them in the first stage, they can be safely ignored in the second one. We note that this is just a possibility, and we are not obliged to do that: as an example, clauses (3-4) will not be processed in the first stage.

Although in general **POS** clauses are not *safe-forget* –because shrinking the extension of R can violate them– we now show that clause (2) it is. In fact, if we equivalently rewrite clauses (2) and (3-4), respectively, as follows:

$$\begin{aligned} \Box X \neg B(X) \Box \neg G(X) \rightarrow R(X) & \quad (2)^{\Box} \\ \Box X R(X) \rightarrow \neg B(X) \Box \neg G(X), & \quad (3-4)^{\Box} \end{aligned}$$

we note that clause (2)[□] sets a lower bound for the extension of R , and clauses (3-4)[□] set an upper bound for it; both the lower and the upper bound are $\text{ext}(\neg B(X) \Box \neg G(X))$. If we use $\neg$ in the first stage– clauses (2,5-8) for computing $\text{ext}(R^{\Box})$, then $\neg$ in the second stage– we can safely define $\text{ext}(R)$ as $\text{ext}(R^{\Box}) \cap \text{ext}(\neg B(X) \Box \neg G(X))$, and no constraint will be violated (cf. Fig. 3). The next theorem shows that is not by chance that the antecedent of (2)[□] is semantically related to the consequence of (3-4)[□].

Theorem 1. Let Φ be an ESO formula of the form:

$$\Box S_1, \dots, S_h, R \equiv \Box \Box X \alpha(X) \rightarrow R(X) \Box \Box X R(X) \rightarrow \beta(X), \quad (9)$$

where Ξ is a conjunction of clauses, both α and β are arbitrary formulae in which R does not occur and X is the only free variable, and it holds that:

Hyp 1: R either does not occur or occurs negatively in Ξ ;

Hyp 2: $\models \Box X \alpha(X) \rightarrow \beta(X)$.

Let $\Phi^{\Box}$ be:

$$\Box S_1, \dots, S_h, R^{\Box} \equiv \Box \Box X \alpha(X) \rightarrow R^{\Box}(X),$$

where $R^\square$ is a new predicate symbol, and $\Xi^\square$ is Ξ with R replaced by $R^\square$. Let Φ^d be:

$$\square X R(X) \leftrightarrow R^\square(X) \square \beta(X).$$

For each database D and each list M^s of extensions for $(S_1, \dots, S_n, R^\square)$ such that $(D, M^s) \models \Phi^s$, then $(D, M^s - \text{ext}(R^\square), \text{ext}(R)) \models \Phi$, where $\text{ext}(R)$ is the extension of R as defined by M^s and Φ^d .

Referring to Fig. 2, Φ is the spec, D is the instance, Φ^s is the “simplified spec”, and $\square X R(X) \rightarrow \beta(X)$ is the “delayed constraint”. Solving Φ^s against D produces –if the instance is satisfiable– a list of extensions M^s (the “output”). Evaluating Φ^d against M^s corresponds to the “PostProcessing” phase in the second stage; since the last stage amounts to the evaluation of a first-order formula against a fixed database, it can be done in logarithmic space (thus in polynomial time).

In other words, the theorem says that, for each satisfiable instance D of the simplified specification Φ^s , each solution M^s of Φ^s can be translated, via Φ^d , to a solution of the original specification Φ ; we can also say that $\Xi \square \square X \alpha(X) \rightarrow R(X)$ is safe-forget, and $\square X R(X) \rightarrow \beta(X)$ is safe-delay.

Referring to the specification of Example 1, Ξ is the conjunction of clauses (5-8), and $\alpha(X)$ and $\beta(X)$ are both $\neg B(X) \square \neg G(X)$, cf. clauses (3-4)¹. Fig. 3 represents possible extensions of the red predicate in the first ($R^\square$) and second (R) stages, for the instance of Fig. 1.

Proofs are omitted for lack of space. As evidence for the soundness of Theorem 1, we claim that Fig. 4 (left) shows that, if Hyp 2 holds, then the constraint $\square X \alpha(X) \rightarrow R(X)$ can never be violated in the second stage.

We are guaranteed that the two-stage process preserves at least one solution of Φ by the following proposition.

Proposition 1. Let D , Φ , Φ^s and Φ^d as in Theorem 1. For each database D , if Φ is satisfiable, Φ^s and Φ^d are satisfiable.

To substantiate the reasonableness of the two hypotheses of Theorem 1, we play the devil’s advocate and add to the specification of Example 1 the constraint

$$\square X \text{ edge}(X, X) \rightarrow R(X), \quad (10)$$

saying that self-loops must be red. We immediately notice that now clauses (3-4) are not safe-delay: intuitively, after the first stage, nodes may be red either because of (2) or because of (10), and (3-4) are not enough to set the correct color for a node. Now, if –on top of (5-8)– Ξ contains also the constraint (10), Hyp 1 is clearly not satisfied. Analogously, if (10) is used to build $\alpha(X)$, then $\alpha(X)$ becomes $\text{edge}(X, X) \square (\neg B(X) \square \neg G(X))$, and Hyp 2 is not satisfied. Fig. 4, right, gives further evidence that the constraint $\square X \alpha(X) \rightarrow R(X)$ can be violated if $\text{ext}(R)$ is computed using Φ^d and $\text{ext}(\alpha)$ is not a subset of $\text{ext}(\beta)$.

Some further comments about Theorem 1 are in order:

- Ξ does not need to be a conjunction of clauses, but can be any formula such that, from any structure M s.t. $M \models \Xi$, shrinking $\text{ext}(R)$ and keeping everything else fixed we obtain another model of Ξ . As an example, Ξ may contain the conjunct $\exists X R(X) \rightarrow G(X)$.
- Although Hyp 2 calls for a tautology check –which is not decidable in general– we will see in what follows that many specifications satisfy it by design.

3.2 Set of monadic predicates

Theorem 1 can be applied recursively to the specification Φ^s , by focusing on a different guessed predicate, in order to obtain a new simplified specification $(\Phi^s)^s$ and new delayed constraints $(\Phi^s)^d$. Since, by Proposition 1, satisfiability of such formulae is preserved, it is afterwards possible to translate, via $(\Phi^s)^d$, each solution of $(\Phi^s)^s$ to a solution of Φ^s , and then, via Φ^d , to a solution of Φ . The procedure `Reformulate` deals with the general case of a set of guessed predicates: if the input specification Φ is satisfiable, it returns a simplified specification $\overline{\Phi^s}$ and a list of delayed constraints $\overline{\Phi^d}$. Algorithm `SolveByDelaying` gets any solution of $\overline{\Phi^s}$ and translates it, via the evaluation of formulae in the list $\overline{\Phi^d}$ –with LIFO policy– to a solution of Φ .

Algorithm `SolveByDelaying`

Input a spec Φ , a database D

Output a solution of $[D, \Phi]$, if satisfiable; ‘unsatisfiable’ o.w.;

```

begin
   $[\overline{\Phi^s}, \overline{\Phi^d}] = \text{REFORMULATE}(\Phi)$ ;
  if  $([\overline{\Phi^s}, D]$  is satisfiable) then
    begin
      let  $M$  be a solution of  $[\overline{\Phi^s}, D]$ ;
      while  $(\overline{\Phi^d}$  is not empty) do
        begin
          Constraint  $d = \overline{\Phi^d}.\text{pop}()$ ;
           $M = M \sqcup$  solution of  $d$ ;
          // cf. Theorem 1
        end;
      return  $M$ ;
    end;
  else return ‘unsatisfiable’;
end;
```

Procedure `Reformulate`

Input a specification Φ

Output $[\overline{\Phi^s}, \overline{\Phi^d}]$, where $\overline{\Phi^s}$ a simplified spec, and $\overline{\Phi^d}$ a stack of delayed constraints

```

begin
  Stack  $\overline{\Phi^d} =$  the empty stack;
   $\overline{\Phi^s} = \Phi$ ;
  for each monadic guessed pred.  $R$  in  $\overline{\Phi^s}$  do
    begin
      partition constr. in  $\overline{\Phi^s}$  according to Thm 1, in:
       $[\Xi; \exists X \alpha(X) \rightarrow R(X); \exists X R(X) \rightarrow \beta(X)]$ 
      if (prev. step is possible with  $\exists X \beta(X) \models \text{TRUE}$ )
        then begin
           $\overline{\Phi^d}.\text{push}(' \exists X R(X) \leftrightarrow R(X) \sqcup \beta(X) ');$ 
           $\overline{\Phi^s} = \Xi \sqcup \exists X \alpha(X) \rightarrow R(X)$ ;
        end;
    end;
  return  $[\overline{\Phi^s}, \overline{\Phi^d}]$ ;
end;
```

As an example, we evaluate the procedure `Reformulate` on the specification of Example 1, by focusing on the guessed predicates in the order R, G, B . The output is the following simplified specification $\overline{\Phi^s}$:

$$\exists R \sqcup G \sqcup B \quad \exists X R(X) \sqcup G(X) \sqcup B(X) \quad \square$$

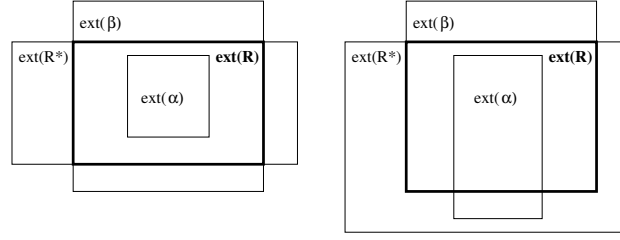


Fig. 4. Extensions with and without Hyp 2

$$\Box X \ Y \ X \equiv Y \ \Box R^{\Box}(X) \ \Box R^{\Box}(Y) \rightarrow \neg \text{edge}(X, Y) \ \Box$$

$$\Box X \ Y \ X \equiv Y \ \Box G^{\Box}(X) \ \Box G^{\Box}(Y) \rightarrow \neg \text{edge}(X, Y) \ \Box$$

$$\Box X \ Y \ X \equiv Y \ \Box B(X) \ \Box B(Y) \rightarrow \neg \text{edge}(X, Y),$$

and the following list $\overline{\Phi^d}$ of delayed constraints:

$$\Box X \ R(X) \leftrightarrow R^{\Box}(X) \ \Box \neg G(X) \ \Box \neg B(X); \quad (11)$$

$$\Box X \ G(X) \leftrightarrow G^{\Box}(X) \ \Box \neg B(X). \quad (12)$$

Note that the check that $\Box X \ \beta(X)$ is not a tautology prevents the (useless) delayed constraint $\Box X \ B(X) \leftrightarrow B^{\Box}(X)$ to be pushed in $\overline{\Phi^d}$.

From any solution of $\overline{\Phi^s}$, a solution of Φ is obtained in the while loop of SolveByDelaying by reconstructing first of all the extension for G by formula (12), and then the extension for R by formula (11) (synthesized, respectively, in the second and first iteration of the algorithm). Since each constraint in the stack $\overline{\Phi^d}$ is first-order, the whole while loop is doable in logarithmic space.

We observe that the procedure Ref or mul at e is intrinsically non-deterministic, because of the partition that must be applied to the constraints.

3.3 Binary predicates

The specification of the k -coloring problem using a binary predicate Col –the first argument being the node and the second the color– is as follows (constraints represent, respectively, covering, disjointness, and good coloring).

$$\begin{aligned} &\Box \text{Col} \ \Box X \ \Box Y \ \text{Col}(X, Y) \ \Box \\ &\Box X \ Y \ Z \ \text{Col}(X, Y) \ \Box \text{Col}(X, Z) \rightarrow Y = Z \ \Box \\ &\Box X \ Y \ Z \ X \equiv Y \ \Box \text{Col}(X, Z) \ \Box \text{Col}(Y, Z) \rightarrow \neg \text{edge}(X, Y). \end{aligned} \quad (13)$$

Since the number of colors is finite, it is always possible to unfold the above constraints with respect to the second argument of Col . As an example, if $k = 3$, we obtain –up to an appropriate renaming of the Col predicate– the specification of Example 1.

The above considerations imply that we can use the architecture of Fig. 2 for a large class of specifications.

Definition 1. A **safe-delay functional spec** is an ESO formula of the form:

$$\Box P \equiv \Box \Box X \Box Y P(X, Y) \Box \Box X Y Z P(X, Y) \Box P(X, Z) \rightarrow Y = Z,$$

where $\equiv$ is a conjunction of clauses in which P either does not occur or occurs negatively.

The term “functional” originates from covering and disjointness constraints, which imply a search space which is a (total) function from a finite domain to a finite codomain. In particular, the disjointness constraints are safe-delay, while the covering and the remaining ones, i.e., $\equiv$, are safe-forget. Formally, soundness of the architecture on safe-delay functional formulae is guaranteed by Theorem 1 and algorithm SolveByDelaying.

Safe-delay functional specifications are quite common; apart from graph coloring, notable examples (formal details are omitted for lack of space) are Bin packing [15] (which goal is to pack n objects of given sizes into k bins of capacity c) and Job shop scheduling [13, Prob. SS18]: there are n jobs, m tasks, and p processors. Jobs are ordered collections of tasks and each task has an integer-valued length and the processor that performs it. Each processor can perform a task at the time, and the tasks belonging to the same job must be performed in their order. Finally, there is a global deadline D that has to be met by all jobs. The disjointness constraint imposes at most one starting time for each task: thus, by applying Theorem 1 for delaying it, we allow a task to have multiple starting times, i.e., the task does not overlap with any other task at any of its start times. In the PostProcessing stage we can arbitrarily choose one of them to obtain a solution of the original problem.

4 Methodological discussion

In this section we make a discussion on the methodology we adopted in this work, in particular the use of ESO as a modelling language, and the choice of the solvers for the preliminary experimentation.

Using ESO for specifying problems wipes out many aspects of state-of-the-art languages which are somehow difficult to take into account (e.g., numbers, arithmetics, constructs for functions, etc.), thus simplifying the task of finding criteria for reformulating problem specifications. However, it must be observed that ESO, even if somewhat limited, is not too far away from the modelling languages provided by some commercial systems. An example of such a language is *ampl*, which admits only linear constraints: in this case, the reformulation technique described in Theorem 1 can often be straightforwardly applied; for instance, a reasonable specification of the k -coloring problem in such a language is the following:

```
param n_nodes; param n_colors integer, > 0;
set NODES := 1..n_nodes; set EDGES within NODES cross NODES;
set COLORS := 1..n_colors;
var Coloring {NODES,COLORS} binary; # Coloring as a 2-ary predicate
```

```

s.t. CoveringAndDisjointness {x in NODES}:
    sum {c in COLORS} Coloring[x,c] = 1; # nodes have exactly one color
s.t. GoodColoring {(x,y) in EDGES, c in COLORS}:
    Coloring[x,c] + Coloring[y,c] <= 1; # nodes linked by an edge have dif. colors

```

The above specification is similar to (13), and the reformulated spec can be obtained by rewriting the “CoveringAndDisjointness” constraint in the following way:

```

s.t. Covering {x in NODES}
    sum {c in COLORS} Coloring[x,c] >= 1;

```

As for languages that admit non linear constraints, e.g., *opl*, it is possible to write a different specification using integer variables for the colors and inequality of colors between adjacent nodes. In this case it is not possible to separate the disjointness constraint from the other ones, since it is implicit in the definition of the domains.

For what concerns the experimentation, it must be noted that a specification written in ESO naturally leads to a translation into a SAT instance. For this reason, we have chosen to use a SAT solver to start the experimentation of the proposed technique. Anyway, since it is well-known that state-of-the-art linear and constraint programming systems may perform much better than SAT on some problems, we repeated the experimentation using the commercial systems *cplex* and *solver*. Thus, SAT-based experiments have to be considered as a starting point and an encouraging evidence of the reasonableness of the proposed approach, by showing that consistent speed-ups in the solving process can be obtained by a mere reformulation of a pure declarative specification. Actually, this evidence has been partially confirmed by the experiments done using the linear integer programming solver *cplex*: also in this case, several classes of instances benefit from our reformulation technique.

5 Experimental results

We made an experimentation of our reformulation techniques on 3-coloring (randomly generated instances), *k*-coloring (instances taken from the DIMACS benchmark repository [ftp://dimacs.rutgers.edu/pub/challenge](http://dimacs.rutgers.edu/pub/challenge)), and job shop scheduling (benchmark instances taken from the OR library at www.ms.ic.ac.uk/info.html).

We solved each instance both with and without delaying the disjointness constraints, using both the DPLL-based SAT system *sat z* [14] (and an ad hoc program [4] for the instantiation stage), and the state-of-the-art constraint and linear programming system *opl* [17], obviously using it as a pure modeling language, and omitting search procedures. For what concerns the latter system, we wrote both a linear and a non-linear specification for the above problems, and applied our reformulation technique to the linear ones (cf. Sec. 4)

Experiments were executed on an Intel 2.4 GHz Xeon bi-processor computer. The size of instances was chosen so that our machine is able to solve (most

of) them in more than few seconds, and less than one hour. In this way, both instantiation and postprocessing, i.e., evaluation of delayed constraints, times are negligible, and comparison can be done only on solving time.

Summing up, we solved several thousands of instances. For what concerns the SAT experimentation, it is worth noting that in all instances the SAT time without disjointness is less than or equal to the time with disjointness. As far as cplex is concerned, we found that several (but not all) instances benefitted from delaying constraints, especially those for which the linear specification is more efficient than the non-linear one (cf. the following paragraphs).

In what follows, we refer to the saving percentage, defined as the ratio

$$(\text{time_with_disjointness} - \text{time_without_disjointness}) / \text{time_with_disjointness}.$$

3-coloring We solved the problem on 3,500 randomly generated graph instances with 430 nodes each. The number of edges varies, and covers the phase transition region [6]: the ratio (# of directed edges / # of nodes) varies between 2 and 6. The average solving time (150 instances for each fixed number of edges) varies between fractions of a second and a minute. The saving percentage in the SAT-based experiments varies between 15% and 50%, for both positive and negative instances, the hardest instances being at 30%. Experimentation with cplex and solver is planned.

k-coloring Results of our SAT-based experiments are shown in Table 1 (n is the number of nodes and e of the edges). The saving percentage varies between 9.0% and 59%.

On the other hand, when applying the reformulation technique to opl, we observed two major evidences:

1. It is not the case that a specification (linear or non-linear) is always more efficient than the other one. In particular, the ratio between the solving time of cplex and solver is highly variable, and the linear specification can be much more efficient than the non-linear one, especially for negative instances.
2. By focusing on the class of instances for which the linear specification is more efficient than the non-linear one, we found out several instances in which delaying the disjointness constraint leads to appreciable time savings, as Table 1 shows.

Job shop scheduling We considered two instances known as FT06 (36 tasks, 6 jobs, 6 processors, solvable with deadline 55) and LA02 (50 tasks, 10 jobs, 5 processors, solvable with deadline 655). SAT solving times are listed in Table 2 for different values for the deadline. As it can be observed, the saving is quite consistent.

For what concerns the experiments in opl, cplex seems not to be affected much by delaying constraints (or even affected negatively), and anyway it is slower than solver.

Summing up, delaying of constraints seems to be always effective when using a SAT solver. As far as cplex is concerned, we have mixed evidence. We plan

Table 1. Solving times (seconds) for k-coloring

Instance	Col- ors	Solv- able?	SAT			CPLEX			SOLVER
			W/ disj.	W/o disj.	% sav.	W/ disj.	W/o disj.	% sav.	
le450_5d	5	Y	6.0	5.2	13.3	540.8	628.1	- 16.1	1.2
le450_15a	13	N	-	-	-	8.6	6.3	26.7	-
le450_25a	21	N	-	-	-	83.6	17.2	79.4	-
DSJC125.9	21	N	-	-	-	1408.4	936.7	33.5	-
DSJC250.1	9	Y	1.6	0.9	43.8	-	-	-	12.5
DSJR500.1	11	N	-	-	-	2.2	1.3	40.9	297.4
DSJR500.1	12	Y	-	-	-	18.4	18.1	1.6	0.5
queen8.8	9	Y	1.5	1.2	20.0	-	2411.2	> 33.0	1.9
queen9.9	10	Y	32.1	25.3	21.2	-	-	-	134.8
queen10.10	15	Y	1.3	0.9	30.8	3.3	3.1	6.1	0.9
queen11.11	13	Y	22.9	17.7	22.7	-	125.1	> 96.5	41.7
queen12.12	15	Y	1.3	0.8	38.5	463.7	228.3	50.8	9.7
fpsol2.i.2	21	N	-	-	-	21.8	13.6	37.6	-

Table 2. Solving times (seconds) for job shop scheduling

Instance	Dead- line	Solv- able?	SAT			CPLEX			SOLVER
			W/ disj.	W/o disj.	% sav.	W/ disj.	W/o disj.	% sav.	
FT06	100	Y	-	4.6	100	117.4	-	- ∞	1.1
FT06	65	Y	3.7	1.7	52.8	-	-	-	1.2
FT06	55	Y	4.0	1.6	60.7	-	-	-	1.5
FT06	54	N	16.1	4.3	73.6	-	-	-	-
FT06	52	N	2.0	1.0	48.0	-	-	-	7.2
LA02	1200	Y	4.6	2.1	53.9	31.2	-	- ∞	1.5
LA02	1000	Y	2.9	1.5	48.8	-	-	-	1.6
LA02	960	Y	22.2	1.8	91.8	-	-	-	-
LA02	860	Y	740.0	8.5	98.8	-	-	-	-
LA02	840	Y	-	15.1	100	-	-	-	-

('-' means that the solver did not terminate in one hour)

to extend the experimentation to problems in which SAT performs worse than other solvers.

6 Conclusions, related and future work

In this paper we have shown a simple reformulation architecture and proven its soundness for a large class of problems. The reformulation allows to delay the solution of some constraints, which results in faster solving. In this way, we have shown that reasoning on a specification can be very effective.

Related work Several researchers addressed the issue of reformulation of a problem after the instantiation phase. As an example, in [11] it is shown that ab-

abstracting problems by simplifying constraints is useful for finding more efficient reformulations of the original problem; abstraction may require backtracking for finding solutions of the original problem. In our work, we focus on reformulation of the specification, and the approach is backtracking-free: once the first stage is completed, a solution will surely be found by evaluating the delayed constraints.

Analogously, in [19] it is shown how to translate an instantiated CSP into its Boolean form, which is useful for finding different reformulations. Finally, in [7] it is shown how to generate a conjunctive decomposition of an instantiated CSP, by localizing independent subproblems.

Other papers investigate the best way to encode an instance of a problem into a format adequate for a specific solver. As an example, many different ways for encoding graph coloring or permutation problems into SAT have been figured out, cf., e.g., [12, 18]. Conversely, in our work we take a specification-oriented approach.

Finally, we point out that a logic-based approach has also been successfully adopted in the '80s to study the query optimization problem for relational DBs. Analogously to the approach presented in this paper, the query optimization problem has been attacked relying on the query (i.e., the specification) only, without considering the database (i.e., the instance), and it was firstly studied in a formal way using first-order logic (cf., e.g., [2, 5]). In a later stage, the theoretical framework has been translated into rules for the automated rewriting of queries expressed in languages and systems used in real world.

Future work In this paper we have focused on a form of reformulation which partitions the first-order part of a specification. This basic idea can be generalized, as an example by evaluating in both stages of the computation a constraint (as an example (10)), in order to allow reformulation for a larger class of specifications. Even more generally, the second stage may amount to the evaluation of a second-order formula. In the future, we plan –with a more extensive experimentation– to check whether such generalizations are effective in practice. It is also interesting to check how delaying of constraints behave when used with other techniques, e.g., symmetry breaking.

Moreover, we wish to extend our results to the so-called permutation problems, i.e., problems which, on top of the constraints of Definition 1 have also the following constraints:

$$\exists X \exists Y P(Y, X) \quad \exists X \exists Y Z P(Y, X) \quad \exists P(Z, X) \rightarrow Y = Z.$$

However, this kind of specifications do not belong to the class studied in this paper, and a generalization of Theorem 1 is needed to support them. Permutation problems arise frequently in theory and practice: examples are the n -queens, the Hamiltonian circuit, and the code generation for parallel assignments (resp. [13, Prob. GT39, PO6]) problems.

Finally, it is our goal to rephrase the theoretical results into rules for automatically reformulating problem specifications given in much more complex languages, e.g. `ampl` and `opl`, which have higher-level built-in constructs, such as integers and functions.

Acknowledgements This research has been supported by MIUR (Italian Ministry for Instruction, University, and Research) under the FIRB project ASTRO (Automazione dell'Ingegneria del Software basata su Conoscenza).

References

1. S. Abiteboul, R. Hull, and V. Vianu. *Foundations of Databases*. Addison Wesley Publ. Co., Reading, Massachusetts, 1995.
2. S. Y. Aho, A.V. and J. Ullman. Equivalences among relational expressions. *SIAM Journal on Computing*, 2(8):218–246, 1979.
3. E. Börger, E. Grädel, and Y. Gurevich. *The Classical Decision Problem. Perspectives in Mathematical Logic*. Springer, 1997.
4. M. Cadoli and A. Schaerf. Compiling problem specifications into SAT. In *Proceedings of the European Symposium On Programming (ESOP 2001)*, volume 2028 of LNCS, pages 387–401. Springer, 2001.
5. A. K. Chandra and P. M. Merlin. Optimal implementation of conjunctive queries in relational databases. In *Proc. 9th ACM Symp. Theory of Computing (STOC'77)*, pages 77–90. Boulder, CO, USA, 1977.
6. P. Cheeseman, B. Kanefski, and W. M. Taylor. Where the really hard problem are. In *Proc. of IJCAI'91*, pages 163–169, 1991.
7. B. Y. Choueiry and G. Noubir. On the computation of local interchangeability in discrete constraint satisfaction problems. In *Proc. of AAAI'98*, pages 326–333, 1998.
8. T. Eiter, N. Leone, C. Mateis, G. Pfeifer, and F. Scarcello. The KR system dl v: Progress report, Comparisons and Benchmarks. In *Proc. of KR'98*, pages 406–417, 1998.
9. R. Fagin. Generalized First-Order Spectra and Polynomial-Time Recognizable Sets. In R. M. Karp, editor, *Complexity of Computation*, pages 43–74. AMS, 1974.
10. R. Fourer, D. M. Gay, and B. W. Kernigham. *AMPL: A Modeling Language for Mathematical Programming*. International Thomson Publishing, 1993.
11. E. C. Freuder and D. Sabin. Interchangeability supports abstraction and reformulation for multi-dimensional constraint satisfaction. In *Proc. of AAAI'97*, pages 191–196, 1997.
12. A. M. Frisch and T. J. Peugniez. Solving non-boolean satisfiability problems with stochastic local search. In *Proc. of IJCAI 2001*, pages 282–290, 2001.
13. M. R. Garey and D. S. Johnson. *Computers and Intractability—A guide to NP-completeness*. W.H. Freeman and Company, San Francisco, 1979.
14. C. M. Li and Anbulagan. Heuristics based on unit propagation for satisfiability problems. In *Proc. of IJCAI'97*, pages 366–371, 1997.
15. S. Martello and P. Toth. *Knapsack Problems: algorithms and computer implementation*. John Wiley & Sons Ltd, 1990.
16. I. Niemelä. Logic programs with stable model semantics as a constraint programming paradigm. *Ann. of Mathematics and Artificial Intelligence*, 25(3,4):241–273, 1999.
17. P. Van Hentenryck. *The OPL Optimization Programming Language*. The MIT Press, 1999.
18. T. Walsh. Permutation problems and channelling constraints. In *Proc. of LPAR 2001*, number 2250 in LNCS, pages 377–391. Springer, 2001.
19. R. Weigel and C. Bliet. On reformulation of constraint satisfaction problems. In *Proc. of ECAI'98*, pages 254–258, 1998.